



TEMA IV

DESARROLLO DE APLICACIONES WEB EN BACKEND

*"Siempre parece imposible,
hasta que se hace"*

Nelson Mandela

COOKIES, SESIONES

El protocolo HTTP se diseñó como un protocolo *sin estado*, cada petición y respuesta está totalmente aislada de las demás. No hay persistencia entre una petición y la siguiente, y ninguno de los atributos de la petición (dirección IP, identificador del agente, etc...) nos permite discriminar de forma segura y consistente las peticiones de una persona de las del resto.

Cookies

Los desarrolladores de navegadores hace tiempo que se dieron cuenta de que esta carencia de estados iba a representar un problema para los desarrolladores web, y así fue como nacieron las cookies(literalmente galleta). Una cookie es una pequeña cantidad de información que el servidor delega en el navegador, de forma que éste la almacena. Cada vez que el cliente web solicita una página del servidor, se le envía de vuelta la cookie.

Cuando abrimos nuestro navegador y escribimos `google.com`, el navegador envía una solicitud HTTP a Google que empieza más o menos así:

```
GET / HTTP/1.1
```

```
Host: google.com
```

```
...
```

Cuando Google responde, la respuesta contiene algo parecido a esto:

```
HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: PREF=ID=5b14f22bdaf1e81c:TM=1167000671:LM=1167000671;
            expires=Sun, 17-Jan-2038 19:14:07 GMT;
            path=/; domain=.google.com
Server: GWS/2.1
...
```

El navegador almacenará el valor indicado

(PREF=ID=5b14f22bdaf1e81c:TM=1167000671:LM=1167000671)

y se lo volverá a enviar a Google cada vez que vuelva a acceder a alguna de sus páginas; de esa forma, la próxima vez que vuelvas a Google, la petición que enviará el navegador se parecerá a esta:

```
GET / HTTP/1.1
Host: google.com
Cookie: PREF=ID=5b14f22bdaf1e81c:TM=1167000671:LM=1167000671
...
```

Google puede saber ahora, gracias al valor de la Cookie, que eres la misma persona que accedió un rato antes. Este valor puede ser, por ejemplo, una clave en una tabla de la base de datos que almacene los datos del usuario. Con esa información, Google puede hacer aparecer tu nombre en la página (de hecho, lo hace).

A la hora de utilizar las capacidades de persistencia de Django, les mejor usar las prestaciones de alto nivel para la gestión de sesiones y de usuarios que veremos más adelante. Aún así, veremos como leer y definir cookies a bajo nivel.

Obtener los valores de las cookies que ya están definidas es muy fácil. Cada objeto de tipo petición, request, contiene un objeto COOKIES que se comporta como un diccionario; puedes usarlo para leer cualquier cookie que el navegador haya enviado a la vista:

```
def show_color(request):
    if "favorite_color" in request.COOKIES:
        return HttpResponse("Your favorite color is %s" % \
            request.COOKIES["favorite_color"])
    else:
        return HttpResponse("You don't have a favorite color.")
```

Definir los valores de las cookies tampoco es muy complicado. Debes usar el método set_cookie() en un objeto de tip HttpResponse.

```
def set_color(request):
    if "favorite_color" in request.GET:
        # Create an HttpResponse object...
        response = HttpResponse("Your favorite color is now %s" % \
            request.GET["favorite_color"])

        # ... and set a cookie on the response
        response.set_cookie("favorite_color",
            request.GET["favorite_color"])

        return response
    else:
        return HttpResponse("You didn't give a favorite color.")
```

Hay una serie de parámetros opcionales que puedes pasar a `response.set_cookie()` y que te permiten controlar determinadas características de la cookie,:

- `max_age`: el tiempo (en segundos) que la cookie debe permanecer activa. Si este parámetro es la cookie, desaparecerá automáticamente cuando se cierre el navegador. (valor por defecto: `None`)
- `expires`: la fecha y hora en que la cookie debe expirar. Debe estar en el formato "Wdy, DD-Mth-YY HH:MM:SS GMT". Si se utiliza este parámetro, su valor tiene preferencia sobre el definido mediante `max_age`. (valor por defecto: `None`)
- `path`: la ruta o path para la cual es válida la cookie. Los navegadores solo reenviarán la cookie a las páginas que estén en dicha ruta. Esto impide que se envíe esta cookie a otras secciones de la web. Es especialmente útil si no se tiene el control del nivel superior de directorios del servidor web. (valor por defecto: `"/"`)
- `domain`: el dominio para el cual es válida la cookie. Se puede usar este parámetro para definir una cookie que sea apta para varios dominios. Por ejemplo, definiendo `domain=".example.com"` la cookie será enviada a los dominios `www.example.com`, `www2.example.com` y aun.otro.subdominio.example.com. Si a este parámetro no se le asigna ningún valor, la cookie solo será enviada al dominio que la definió. (valor por defecto: `"/"`)
- `secure`: si este valor se define como `True`, se le indica al navegador que sólo retorne esta cookie a las páginas que se accedan de forma segura (protocolo HTTPS en vez de HTTP). (valor por defecto: `False`)

Puede que te hayas dado cuenta de algunos de los problemas potenciales que se presentan con esto de las cookies; veamos algunos:

1. El almacenamiento de los cookies es opcional; los navegadores no dan ninguna garantía. De hecho, los navegadores permiten al usuario definir una política de aceptación o rechazo de las mismas. Para darte cuenta de lo muy usadas que son las cookies en la web actual, simplemente activa la opción de "Avisar antes de aceptar cualquier cookie" y date un paseo por Internet.

A pesar de su uso habitual, las cookies son el ejemplo perfecto de algo que no es confiable. Esto significa que el desarrollador debe comprobar que el usuario está

dispuesto a aceptar las cookies antes de confiar en ellas.

Aún más importante, nunca debes almacenar información fundamental en las cookies.

2. Las Cookies no son seguras (especialmente aquellas que no se envían mediante HTTPS) Dado que los datos enviados viajan en texto claro, están expuestas a que terceras personas lean esa información, lo que se llama ataques de tipo snooping (por snoop, fisgonear, husmear). Nunca se debe almacenar información confidencial en una cookie.

Hay otro tipo de ataque, aún más insidioso, conocido como ataque man-in-the-middle o MitM (ataque de tipo Hombre-en-medio o Intermediario). Aquí, el atacante no solo intercepta la cookie, sino que además la usa para actuar ante el servidor como si fuera el usuario legítimo.

3. Las Cookies ni siquiera son seguras para los servidores. La mayoría de los navegadores permiten manipular y editar de forma sencilla los contenidos de cookies individuales, y existen herramientas como mechanize (<http://wwwsearch.sourceforge.net/mechanize/>) que permiten a cualquiera construir solicitudes HTTP a mano.

Así que tampoco debemos almacenar en las cookies datos que sean fáciles de falsificar.

"El error habitual en este escenario consiste en almacenar algo así como `IsLoggedIn=1` en una cookie cuando el usuario se ha validado. Te sorprendería saber cuantos sitios web cometen este tipo de error; no lleva más de unos segundos engañar a sus sistemas de seguridad".

Django incluye una forma fácil de comprobar que el cliente del usuario disponga de la capacidad de aceptar cookies. Sólo es necesario llamar a la función `request.session.set_test_cookie()` en una vista, y comprobar en otra vista distinta el resultado de llamar a `request.session.test_cookie_worked()`.

Esta división un tanto extraña entre las llamadas a estas dos funciones se debe a la forma en que trabajan las cookies. Cuando se define una cookie, no tienes forma de saber si el navegador la ha aceptado realmente hasta la siguiente solicitud.

Es una práctica recomendable llamar a la función `delete_test_cookie()` para limpiar la cookie de prueba después de haberla usado. Lo mejor es hacerlo justo

después de haber verificado que las cookies funcionan.

Ejemplo

```
def login(request):  
    # If we submitted the form...  
  
    if request.method == 'POST':  
        # Check that the test cookie worked (we set it below):  
  
        if request.session.test_cookie_worked():  
            # The test cookie worked, so delete it.  
            request.session.delete_test_cookie()  
  
            # In practice, we'd need some logic to check username/password  
            # here, but since this is an example...  
  
            return HttpResponse("You're logged in.")  
  
            # The test cookie failed, so display an error message. If this  
            # was a real site we'd want to display a friendlier message.  
        else:  
            return HttpResponse("Please enable cookies and try again.")  
  
    # If we didn't post, send the test cookie along with the login form.  
    request.session.set_test_cookie()  
  
    return render_to_response('foo/login_form.html')
```

SESIONES

Con todas estas limitaciones y agujeros potenciales de seguridad, la gestión de las cookies y de las sesiones persistentes es el origen de muchos problemas para los desarrolladores web. Uno de los objetivos de Django es evitar eficazmente ésto, así que dispone de un entorno de sesiones diseñado para suavizar y facilitar todas estas cuestiones.

El entorno de sesiones permite almacenar y recuperar cualquier dato basándote en la sesión del usuario. Almacena la información relevante solo en el servidor y abstrae todo el problema del envío y recepción de las cookies. Estas solo almacenan una versión codificada (hash) del identificador de la sesión, y ningún otro dato.

Veamos como activar las sesiones, y como usarlas en nuestras vistas.

Las sesiones se implementan mediante un poco de middleware y un modelo Django.

Para activar las sesiones, necesitas que :

1. El valor de `MIDDLEWARE_CLASSES` contenga
`'django.contrib.sessions.middleware.SessionMiddleware'.`
2. En las `INSTALLED_APPS`
`'django.contrib.sessions'` esté incluido

Por defecto al ejecutar `startproject` ya tienes estas dos características habilitadas, así que a menos que las hayas borrado, es muy probable que no tengas que hacer nada para empezar a usar las sesiones.

Si lo que quieres es no usar sesiones, deberías quitar la referencia a `SessionMiddleware` de `MIDDLEWARE_CLASSES` y borrar `'django.contrib.sessions'` de las app instaladas

Cuando están activadas las sesiones, los objetos `HttpRequest` tendrán un atributo llamado `session`, que se comporta igual que un diccionario. Se puede leer y escribir en él de la misma forma en que lo harías con un diccionario normal.

```
# Set a session value:
request.session["fav_color"] = "blue"

# Get a session value -- this could be called in a different view,
# or many requests later (or both):
fav_color = request.session["fav_color"]

# Clear an item from the session:
del request.session["fav_color"]

# Check if the session has a given key:
if "fav_color" in request.session:
    ...
```

También puedes usar otros métodos propios de un diccionario como `keys()` o `items()` en `request.session`.

Hay dos o tres reglas muy sencillas para usar eficazmente las sesiones en Django:

- Debes usar sólo cadenas de texto normales como valores de clave en `request.session`, en vez de, por ejemplo, enteros, objetos, etc. Esto es más un convenio que un regla en el sentido estricto, pero merece la pena seguirla.
- Los valores de las claves de una sesión que empiecen con el carácter subrayado están reservadas para uso interno de Django. En la práctica, sólo hay unas pocas variables así, pero, a no ser que sepas lo que estás haciendo (y estés dispuesto a mantenerte al día en los cambios internos de Django), lo mejor que puedes hacer es evitar usar el carácter subrayado como prefijo en tus propias variables; eso impedirá que Django pueda interferir con tu aplicación,
- Nunca reemplaces `request.session` por otro objeto, y nunca accedas o modifiques sus atributos. Utilízalo sólo como si fuera un diccionario.

Ejemplo:

Esta vista simplificada define una variable `has_commente` como `True` después de que el usuario haya publicado un comentario. Es una forma sencilla (aunque no

particularmente segura) de impedir que el usuario publique dos veces el mismo comentario:

```
def post_comment(request, new_comment):
    if request.session.get('has_commented', False):
        return HttpResponse("You've already commented.")
    c = comments.Comment(comment=new_comment)
    c.save()
    request.session['has_commented'] = True
    return HttpResponse('Thanks for your comment!')
```

El método get busca en el diccionario esa clave, si la encuentra nos retorna su correspondiente valor y si no está retorna False. De forma menos abreviada ese código es equivalente a:

```
if "has_commented" in request.session:
    result = request.session["has_commented"]
else:
    result = False
if result:
```

Ejemplo:

Esta vista permite que un usuario se identifique como tal en nuestras páginas:

```
def login(request):
    try:
        m = Member.objects.get(username__exact=request.POST['username'])
        if m.password == request.POST['password']:
            request.session['member_id'] = m.id
```

```
        return HttpResponse("You're logged in.")

    except Member.DoesNotExist:

        return HttpResponse("Your username and password didn't match.")
```

Y ésta permite cerrar o salir de la sesión:

```
def logout(request):

    try:

        del request.session['member_id']

    except KeyError:

        pass

    return HttpResponse("You're logged out.")
```

El mecanismo de autenticación que presentaremos un poco más adelante realiza esta tarea de forma mucho más segura y robusta. Estos ejemplos son para comprender el funcionamiento de las sesiones

Internamente, cada sesión es simplemente un modelo de entidad de Django como cualquier otro, definido en `django.contrib.sessions.models`. Cada sesión se identifica gracias a un hash pseudo-aleatorio de 32 caracteres

```
>>> from django.contrib.sessions.models import Session
>>> s = Session.objects.get(pk='2b1189a188b44ad18c35e113ac6ceead')
>>> s.expire_date
datetime.datetime(2005, 8, 20, 13, 35, 12)
```

Para poder acceder a los datos de la sesión, hay que usar el método `get_decoded()`. Esto se debe a que estos datos, que consistían en un diccionario, están almacenados codificados:

```
>>> s.session_data
'KGRwMQpTJJ19hdXRox3VzZXJfaWQnCnAyCkxkCnMuMTExY2ZjODI2Yj...'
```

```
>>> s.get_decoded()  
{'user_id': 42}
```

Django, en principio, solo almacena la sesión en la base de datos si ésta ha sido modificada; es decir, si cualquiera de los valores almacenados en el diccionario es asignado o borrado. Esto puede dar lugar a algunos errores sutiles, como se indica en el último ejemplo:

```
# Session is modified.  
request.session['foo'] = 'bar'  
  
# Session is modified.  
del request.session['foo']  
  
# Session is modified.  
request.session['foo'] = {}
```

Se puede cambiar este comportamiento, especificando la opción `SESSION_SAVE_EVERY_REQUEST` a `True`. Si lo hacemos así, Django almacenará la sesión en la base de datos en cada petición, incluso si no se ha modificado ninguno de sus valores.

La cookie de sesión sólo se envía cuando se ha creado o modificado una sesión. Si `SESSION_SAVE_EVERY_REQUEST` está como `True`, la cookie de sesión será reenviada en cada petición. De forma similar, la sección de expiración (`expires`) se actualizará cada vez que se reenvíe la cookie.

Además de las características ya mencionadas, hay otros valores de configuración que influyen en la gestión de sesiones con Django, tal y como se muestra en la siguiente lista:

- `SESSION_COOKIE_DOMAIN`: el Dominio a utilizar por la cookie de sesión. Se puede utilizar, por ejemplo, el valor `".lawrence.com"` para utilizar la cookie en diferentes subdominios. El valor `None` indica una cookie estándar. (Valor por defecto: `None`).

- `SESSION_COOKIE_NAME`: el nombre de la cookie de sesiones. Puede ser cualquier cadena de texto. (Valor por defecto: "sessionid")
- `SESSION_COOKIE_SECURE`: indica si se debe usar una cookie segura para la cookie de sesión. Si el valor es `True`, la cookie se marcará como segura, lo que significa que sólo se podrá utilizar mediante el protocolo HTTPS. (Valor por defecto: `False`).

Los datos de la sesión se almacenan en una tabla en la base de datos llamada `django_session`.

Los datos de la sesión son suministrados bajo demanda. Si nunca accedes al atributo `request.session`, Django nunca accederá a la base de datos.

Django sólo envía la cookie si tiene que hacerlo. Si no modificas ningún valor de la sesión, no reenvía la cookie (a no ser que hayas definido `SESSION_SAVE_EVERY_REQUEST` como `True`).

El entorno de sesiones de Django se basa entera y exclusivamente en las cookies. No almacena la información de la sesión en las URL, como recurso extremo en el caso de que no se puedan utilizar las cookies, como hacen otros entornos (PHP, JSP).

Las sesiones nos permiten almacenar información a lo largo de las diferentes peticiones del navegador; Utilizamos esas sesiones para validar al usuario, es decir, permitirle hacer login. No podemos confiar en que los usuarios sean quien dicen ser, necesitamos autentificarlos de alguna manera.

Django nos proporciona las herramientas necesarias para tratar con este problema. El sistema de autenticación de usuarios de Django maneja cuentas de usuarios, grupos, permisos y sesiones basadas en cookies (autenticación y autorización). El nombre implica que, a menudo, tratar con los usuarios implica dos procesos. Se necesita:

- Verificar (autenticación) que un usuario es quien dice ser (Normalmente comprobando un nombre de usuario y una contraseña contra una tabla de una base de datos)
- Verificar que el usuario está autorizado (autorización) a realizar una operación determinada (normalmente comprobando una tabla de permisos)

Siguiendo estos requerimientos, el sistema aut/aut de Django consta de los siguientes componentes:

- Usuarios: Personas registradas en tu sitio web
- Permisos: Valores binarios (Si/No) que indican si un usuario puede o no realizar una tarea determinada.
- Grupos: Una forma genérica de aplicar etiquetas y permisos a más de un usuario.
- Mensajes: Un mecanismo sencillo que permite enviar y mostrar mensajes del sistema usando una cola.
- Perfiles: Un mecanismo que permite extender los objetos de tipo usuario con campos adicionales.

Hemos visto estas utilidades en el panel de administración

Autenticación

Al igual que ocurría con las sesiones, el sistema de autenticación viene incluido como una aplicación en el módulo `django.contrib`. De igual manera, viene instalado por defecto

La principal interfaz que usarás para trabajar con los datos del usuario dentro de una vista es `request.user`; es un objeto que representa al usuario que está conectado en ese momento. Si no hay ningún usuario conectado, este objeto será una instancia de la clase `AnonymousUser`

Puedes saber fácilmente si el usuario está identificado o no con el método `is_authenticated()`:

```
if request.user.is_authenticated():  
    # Do something for authenticated users.  
  
else:  
    # Do something for anonymous users.
```

Una vez que ya tienes un usuario dispondrás de una serie de campos de datos y métodos asociados al mismo. Hay que comprobar el resultado de `user.is_authenticated()` antes de asumir de buena fe que el usuario es legítimo.

Campos de los objetos User:

- `username` (obligatorio) 30 caracteres como máximo. Sólo acepta caracteres alfanuméricos (letras, dígitos y el carácter subrayado).
- `first_name` (opcional) 30 caracteres como máximo.
- `last_name` (opcional) 30 caracteres como máximo.
- `Email` (opcional). Dirección de correo electrónico.
- `Password` (obligatorio). Un código de comprobación (hash), junto con otros metadatos de la contraseña. Django nunca almacena la contraseña en crudo. Véase la sección "Cambia contraseñas" para más información.
- `is_staff` (booleano). Indica que el usuario puede acceder a las secciones de administración.
- `is_active` (booleano). Indica que la cuenta puede ser usada para identificarse. Se puede poner a `False` para deshabilitar a un usuario sin tener que borrarlo de la tabla.
- `is_superuser` (booleano). Señala que el usuario tiene todos los permisos, aún cuando no se le hayan asignado explícitamente.
- `last_login`. Fecha y hora de la última vez que el usuario se identificó. Se asigna automáticamente a la fecha actual por defecto.
- `date_joined`. Fecha y hora en que fue creada esta cuenta de usuario. Se asigna automáticamente a la fecha actual en su momento.

Métodos de los objetos User:

- `is_authenticated()`: siempre devuelve `True` para usuarios reales. Es una forma de determinar si el usuario se ha identificado. Esto no implica que posea ningún permiso, y tampoco comprueba que la cuenta esté activa. Sólo indica que el usuario se ha identificado con éxito.
- `is_anonymous()`: devuelve `True` sólo para usuarios anónimos, y `False` para usuarios "reales". En general, es preferible usar el método `is_authenticated()`.

- `get_full_name()`: devuelve la concatenación de los campos `first_name` y `last_name`, con un espacio en medio.
- `set_password(password)`: cambia la contraseña del usuario a la cadena de texto en claro indicada, realizando internamente las operaciones necesarias para calcular el código de comprobación o hash necesario. Este método no guarda el objeto `User`.
- `check_password(password)`: devuelve `True` si la cadena de texto en claro que se le pasa coincide con la contraseña del usuario. Realiza internamente las operaciones necesarias para calcular los códigos de comprobación o hash necesarios.
- `get_group_permissions()`: devuelve una lista con los permisos que tiene un usuario, obtenidos a través del grupo o grupos a los que pertenezca.
- `get_all_permissions()`: devuelve una lista con los permisos que tiene concedidos un usuario, ya sea a través de los grupos a los que pertenece o bien asignados directamente.
- `has_perm(permission)`: devuelve `True` si el usuario tiene el permiso indicado. El valor de `permission` está en el formato "package.codename". Si el usuario no está activo, siempre devolverá `False`.
- `has_perms(permission_list)`: devuelve `True` si el usuario tiene todos los permisos indicados. Si el usuario no está activo, siempre devolverá `False`.
- `has_module_perms(app_label)`: devuelve `True` si el usuario tiene algún permiso en la etiqueta de aplicación indicada, `app_label`. Si el usuario no está activo, siempre devolverá `False`.
- `get_and_delete_messages()`: devuelve una lista de mensajes (objetos de la clase `Message`) de la cola del usuario, y los borra posteriormente.
- `email_user(subject, message)`: envía un correo electrónico al usuario. El mensaje aparece como enviado desde la dirección indicada en el valor `DEFAULT_FROM_EMAIL`. Se le puede pasar un tercer parámetro opcional, `from_email`, para indicar otra dirección de remite distinta.
- `get_profile()`: devuelve un perfil del usuario específico para el sitio. En la sección "Perfiles" se explicará con más detalle este método.

Por último, los objetos de tipo `User` mantienen dos campos de relaciones: grupos y permisos (`group` y `permissions`). Se puede acceder a estos objetos relacionados de la misma manera en que se usan otros campos múltiples:

```
# Set a user's groups:
myuser.groups = group_list

# Add a user to some groups:
myuser.groups.add(group1, group2,...)

# Remove a user from some groups:
myuser.groups.remove(group1, group2,...)

# Remove a user from all groups:
myuser.groups.clear()

# Permissions work the same way
myuser.permissions = permission_list
myuser.permissions.add(permission1, permission2, ...)
myuser.permissions.remove(permission1, permission2, ...)
myuser.permissions.clear()
```

Iniciar y cerrar sesión

Django proporciona vistas predefinidas para gestionar la entrada del usuario, (el momento en que se identifica), y la salida, (es decir, cuando cierra la sesión). Veamos como hacer que los usuario puedan iniciar y cerrar la sesión "a mano". Django incluye dos funciones en el módulo `django.contrib.auth`: `authenticate()` y `login()`.

Para autenticar un identificador de usuario y una contraseña, se utiliza la función `authenticate()`. Esta función acepta dos parámetros , `username` y `password`, y devuelve un objeto de tipo `User` si la contraseña es correcta para el identificador de usuario. Si falla la comprobación la función devolverá `None`.

```
>>> from django.contrib import auth
>>> user = auth.authenticate(username='john', password='secret')

>>> if user is not None:
...     print "Correct!"
... else:
```

```
...     print "Oops, that's wrong!"
```

La llamada a `authenticate()` sólo verifica las credenciales del usuario, para completar el inicio de sesión hay que realizar una llamada a `login()`. Esta función recibe dos parámetros `HttpRequest` y `User` y almacena el identificador del usuario en la sesión, usando el entorno de sesiones de Django.

```
from django.contrib import auth

def login(request):
    username = request.POST['username']
    password = request.POST['password']
    user = auth.authenticate(username=username, password=password)

    if user is not None and user.is_active:
        # Correct password, and the user is marked "active"
        auth.login(request, user)
        # Redirect to a success page.
        return HttpResponseRedirect("/account/loggedin/")
    else:
        # Show an error page
        return HttpResponseRedirect("/account/invalid/")
```

Para cerrar la sesión, se puede llamar a `django.contrib.auth.logout()` dentro de una vista. Necesita que se le pase como parámetro un objeto de tipo `HttpRequest`, y no devuelve ningún valor:

```
from django.contrib import auth

def logout(request):
    auth.logout(request)
    # Redirect to a success page.
```

```
return HttpResponseRedirect("/account/loggedout/")
```

La llamada a `logout()` no produce ningún error, aun si no hubiera ningún usuario conectado.

En la práctica, no es normalmente necesario escribir tus propias funciones para realizar estas tareas; el sistema de autenticación viene con un conjunto de vistas predefinidas para ello.

El primer paso para utilizar las vistas de autenticación es mapearlas en tu `URLconf`. Necesitas modificar tu código hasta tener algo parecido a esto:

```
from django.contrib.auth.views import login, logout

urlpatterns = patterns('',
    # existing patterns here...
    (r'^accounts/login/$', login),
    (r'^accounts/logout/$', logout),
)
```

`/accounts/login/` y `/accounts/logout/` son las URL por defecto que usa Django para estas vistas.

Por defecto, la vista de `login` utiliza la plantilla definida en `registration/login.html`. El formulario necesita contener un campo llamado `username` y otro llamado `password`. Una plantilla de ejemplo podría ser esta:

```
{% extends "base.html" %}

{% block content %}

    {% if form.errors %}

        <p class="error">Sorry, that's not a valid username or password</p>

    {% endif %}

    <form action='.' method='post'>

        <label for="username">User name:</label>
```

```
<input type="text" name="username" value="" id="username">
<label for="password">Password:</label>
<input type="password" name="password" value="" id="password">

<input type="submit" value="login" />
<input type="hidden" name="next" value="{{ next|escape }}" />
<form action='.' method='post'>

{% endblock %}
```

Limitar el acceso a los usuarios identificados

Por supuesto, la razón de haber implementado todo este sistema es permitirnos limitar el acceso a determinadas partes de nuestro sitio.

La forma más simple y directa de limitar este acceso es comprobar el resultado de llamar a la función `request.user.is_authenticated()` y redirigir a una página de identificación, si procede:

```
from django.http import HttpResponseRedirect

def my_view(request):
    if not request.user.is_authenticated():
        return HttpResponseRedirect('/login/?next=%s' % request.path)
    # ...
```

O quizás mostrar un mensaje de error:

```
def my_view(request):
    if not request.user.is_authenticated():
        return render_to_response('myapp/login_error.html')
```

```
# ...
```

Si se desea abreviar, se puede usar el decorador `login_required` sobre las vistas que nos interese proteger:

```
from django.contrib.auth.decorators import login_required
```

```
@login_required
```

```
def my_view(request):
```

```
# ...
```

Esto es lo que hace el decorador `login_required`:

- Si el usuario no está identificado, redirige a la dirección `/accounts/login/`, incluyendo la url actual como un parámetro con el nombre `next`, por ejemplo `/accounts/login/?next=/polls/3/`.
- Si el usuario está identificado, ejecuta la vista sin ningún cambio. La vista puede asumir sin problemas que el usuario está identificado correctamente

Limitar el acceso a usuarios que pasan una prueba

Se puede limitar el acceso basándose en ciertos permisos o en algún otro tipo de prueba, o proporcionar una página de identificación distinta de la vista por defecto, y las dos cosas se hacen de manera similar.

La forma más cruda es directamente en el código de la vista. Por ejemplo, para comprobar que el usuario está identificado y que, además, tenga asignado el permiso `polls.can_vote` haríamos:

```
def vote(request):
```

```
if request.user.is_authenticated() and request.user.has_perm('polls.can_vote')
```

```
:
```

```
    # vote here
```

```
else:
```

```
return HttpResponse("You can't vote in this poll.")
```

De nuevo, Django proporciona una forma abreviada llamada `user_passes_test`. Requiere que se la pasen unos argumentos y genera un decorador especializado para cada situación en particular:

```
def user_can_vote(user):  
    return user.is_authenticated() and user.has_perm("polls.can_vote")  
  
@user_passes_test(user_can_vote, login_url="/login/")  
def vote(request):  
    # Code here can assume a logged-in user with the correct permission.  
    ...
```

El decorador `user_passes_test` tiene un parámetro obligatorio: un objeto que se pueda llamar (normalmente una función) y que a su vez acepte como parámetro un objeto del tipo `User`, y devuelva `True` si el usuario puede acceder y `False` en caso contrario. Es importante tener en cuenta que `user_passes_test` no comprueba automáticamente que el usuario esté identificado; esa es una comprobación que se debe hacer explícitamente.

En este ejemplo, hemos usado también un segundo parámetro opcional, `login_url`, que te permite indicar la url de la página que el usuario debe utilizar para identificarse (`/accounts/login/` por defecto).

Comprobar si un usuario posee un determinado permiso es una tarea muy frecuente, así que Django proporciona una forma abreviada para estos casos: El decorador `permission_required()`. Usando este decorador, el ejemplo anterior se podría codificar así:

```
from django.contrib.auth.decorators import permission_required  
@permission_required('polls.can_vote', login_url="/login/")  
def vote(request):  
    # ...
```

El decorador `permission_required()` también acepta el parámetro opcional `login_url`, de nuevo con el valor `/accounts/login/` en caso de omisión.

Gestionar usuarios, permisos y grupos

La forma más fácil de gestionar el sistema de autenticación es a través de la interfaz de administración `admin`. A veces, no obstante, hace falta un mayor control, y para eso podemos utilizar las llamadas a bajo nivel que detallamos a continuación.

Crear usuarios

Puedes crear usuarios con el método `create_user`:

```
>>> from django.contrib.auth.models import User
>>> user = User.objects.create_user(username='john',
...                                email='jlennon@beatles.com',
...                                password='glass onion')
```

En este momento, `user` es una instancia de la clase `User`, preparada para ser almacenada en la base de datos (`create_user()` no llama al método `save()`). Este te permite cambiar algunos de sus atributos antes de guardarlos, si quieres:

```
>>> user.is_staff = True
>>> user.save()
```

Cambia contraseñas

Puedes cambiar las contraseña de un usuario llamando a `set_password()`:

```
>>> user = User.objects.get(username='john')
>>> user.set_password('goo goo goo joob')
>>> user.save()
```

No debes modificar directamente el atributo `password`, a no ser que tengas muy claro lo que estás haciendo. La contraseña se almacena en la base de datos en forma de código de comprobación (salted hash) y, por tanto, debe ser modificada sólo a través de este método.

Para ser más exactos, el atributo `password` de un objeto `User` es una cadena de texto con el siguiente formato:

```
hashtype$salt$hash
```

El valor de `hashtype` puede ser `sha1` (por defecto) o `md5`, el algoritmo usado para realizar una transformación hash de un solo sentido sobre la contraseña. El grano de sal es una cadena de texto aleatoria que se utiliza para aumentar la resistencia de esta codificación frente a un ataque por diccionario. Por ejemplo:

```
sha1$a1976$a36cc8cbf81742a8fb52e221aaeab48ed7f58ab4
```

Las funciones `User.set_password()` y `User.check_password()` manejan todos estos detalles y comprobaciones de forma transparente.

El alta del usuario

Podemos usar estas herramientas de bajo nivel para crear vistas que permitan al usuario darse de alta. Prácticamente todos los desarrolladores quieren implementar el alta del usuario a su manera, por lo que Django da la opción de crearte tu propia vista para ello. Afortunadamente, es muy fácil de hacer.

La forma más sencilla es escribir una pequeña vista que pregunte al usuario los datos que necesita y con ellos se cree directamente el usuario. Django proporciona un formulario prefabricado que se puede usar con este fin, como se muestra en el siguiente ejemplo:

```
from django import oldforms as forms
from django.http import HttpResponseRedirect
from django.shortcuts import render_to_response
from django.contrib.auth.forms import UserCreationForm
```

```
def register(request):
    form = UserCreationForm()

    if request.method == 'POST':
        data = request.POST.copy()
        errors = form.get_validation_errors(data)
        if not errors:
            new_user = form.save(data)
            return HttpResponseRedirect("/books/")
        else:
            data, errors = {}, {}

    return render_to_response("registration/register.html", {
        'form' : forms.FormWrapper(form, data, errors)
    })
```

Este formulario asume que existe una plantilla llamada registration/register.html. esa plantilla podría consistir en algo parecido a esto:

```
{% extends "base.html" %}

{% block title %}Create an account{% endblock %}

{% block content %}
    <h1>Create an account</h1>
    <form action="." method="post">
        {% if form.error_dict %}
            <p class="error">Please correct the errors below.</p>
        {% endif %}
    </form>
{% endblock %}
```

```
{% endif %}

{% if form.username.errors %}
    {{ form.username.html_error_list }}
{% endif %}
<label for="id_username">Username:</label> {{ form.username }}

{% if form.password1.errors %}
    {{ form.password1.html_error_list }}
{% endif %}
<label for="id_password1">Password: {{ form.password1 }}

{% if form.password2.errors %}
    {{ form.password2.html_error_list }}
{% endif %}
<label for="id_password2">Password (again): {{ form.password2 }}

<input type="submit" value="Create the account" />
</label>

{% endblock %}
```

Usar información de autenticación en plantillas

El usuario actual, así como sus permisos, están disponibles en el contexto de la plantilla cuando usas RequestContext

Cuando se usa `RequestContext`, el usuario actual (ya sea una instancia de `User` o de `AnonymousUser`) es accesible en la plantilla con el nombre `{{ user }}`:

```
{% if user.is_authenticated %}

    <p>Welcome, {{ user.username }}. Thanks for logging in.</p>

{% else %}

    <p>Welcome, new user. Please log in.</p>

{% endif %}
```

Los permisos del usuario se almacenan en la variable `{{ perms }}`. En realidad, es una forma simplificada de acceder a un par de métodos sobre los permisos que veremos en breve.

Hay dos formas de usar este objeto `perms`. Puedes usar `{{ perms.polls }}` para comprobar si un usuario tienen algún permiso para una determinada aplicación, o se puede usar una forma más específica, como `{{ perms.polls.can_vote }}`, para comprobar si el usuario tiene concedido un permiso en concreto.

Por lo tanto, se pueden usar estas comprobaciones en sentencias `{% if %}`:

```
{% if perms.polls %}

    <p>You have permission to do something in the polls app.</p>

    {% if perms.polls.can_vote %}

        <p>You can vote!</p>

    {% endif %}

{% else %}

    <p>You don't have permission to do anything in the polls app.</p>

{% endif %}
```

Hay unas cuantas cosas que pertenecen al entorno de autenticación y que hasta ahora sólo hemos podido ver de pasada. En esta sección las veremos con un poco más de detalle.

Permisos

Los permisos son una forma sencilla de "marcar" que determinados usuarios o grupos pueden realizar una acción. Se usan normalmente para la parte de administración de Django, pero puedes usarlos también en tu código.

El sistema de administración de Django utiliza los siguientes permisos:

- Acceso a visualizar el formulario "Añadir", y Añadir objetos, está limitado a los usuarios que tengan el permiso add para ese tipo de objeto.
- El acceso a la lista de cambios, ver el formulario de cambios y cambiar un objeto está limitado a los usuarios que tengan el permisos change para ese tipo de objeto.
- Borrar objetos está limitado a los usuarios que tengan el permiso delete para ese tipo de objeto.

Estos tres permisos básicos, añadir, cambiar y borrar, se crean automáticamente para cualquier modelo Django que incluya una clase Admin. Entre bambalinas, los permisos se agregan a la tabla auth_permission cuando ejecutas `manage.py syncdb`.

Estos permisos se crean con el siguiente formato: "<app>.<action>_<object_name>". Por ejemplo, si tienes una aplicación llamada `encuestas`, con un modelo llamado `Respuesta`, se crearan automáticamente los tres permisos con los nombres `"encuestas.add_respuesta"`, `"encuestas.change_respuesta"` y `"encuestas.delete_respuesta"`.

Hay que tener cuidado de que el modelo tenga creada una clase Admin a la hora de ejecutar `syncdb`. Si no la tiene, no se crearán los permisos. Si has inicializado la base de datos y has añadido la clase Admin con posterioridad, debes ejecutar otra vez `syncdb` para crear los permisos.

También puedes definir tus propios permisos para un modelo, usando el atributo `permissions` en la clase Meta. El siguiente ejemplo crea tres permisos hechos a medida:

```
class USCitizen(models.Model):  
    # ...  
    class Meta:
```

```
permissions = (  
    # Permission identifier      human-readable permission name  
    ("can_drive",                "Can drive"),  
    ("can_vote",                "Can vote in elections"),  
    ("can_drink",               "Can drink alcohol"),  
)
```

Esto permisos sólo se crearán cuando ejecutes syncdb. Es responsabilidad tuya comprobar estos permisos en tus vistas.

Igual que con los usuarios, los permisos se implementa en un modelo Django que reside en el módulo `django.contrib.auth.models`. Esto significa que puedes usar la API de acceso a la base de datos para interactuar con los permisos de la forma que quieras.

Grupos

Los grupos son una forma genérica de trabajar con varios usuarios a la vez, de forma que se les pueda asignar permisos o etiquetas en bloque. Un usuario puede pertenecer a varios grupos a la vez.

Un usuario que pertenezca a un grupo recibe automáticamente todos los permisos que se la hayan otorgado al grupo. Por ejemplo, si el grupo `Editores` tiene el permiso `can_edit_home_page`, cualquier usuario que pertenezca a dicho grupo también tiene ese permiso.

Los grupos también son una forma cómoda de categorizar a los usuarios para asignarles una determinada etiqueta, o para otorgarles una funcionalidad extra. Por ejemplo, se puede crear un grupo `Usuarios especiales`, y utilizar código para permitir el acceso a determinadas porciones de tu sitio sólo a los miembros de ese grupo, o para enviarles un correo electrónico sólo a ellos.

Al igual que con los usuarios, la manera más sencilla de gestionar los grupos es usando la interfaz de administración de Django. Los grupos, en cualquier caso, son modelos Django que residen en el módulo `django.contrib.auth.models` así que, al igual que en el caso anterior, puedes usar la API de acceso a la base de datos para trabajar con los grupos a bajo nivel.

Mensajes

El sistema de mensajes es una forma muy ligera y sencilla de enviarle mensajes a un usuario. Cada usuario tiene asociada una cola de mensajes, de forma que los mensajes lleguen en el orden en que fueron enviados. Los mensajes no tienen ni fecha de caducidad ni fecha de envío.

La interfaz de administración de Django usa los mensajes para notificar que determinadas acciones han podido ser llevadas a cabo con éxito. Por ejemplo, al crear un objeto, verás que aparece un mensaje en lo alto de la página de administración, indicando que se ha podido crear el objeto sin problemas.

Puedes usar la misma API para enviar o mostrar mensajes en tu propia aplicación. Las llamadas de la API son bastante simples:

- Para crear un nuevo mensaje usa `user.message_set.create(message='message_text')`.
- Para recuperar/eliminar mensajes usa `user.get_and_delete_messages()`, la cual retorna una lista de objetos `Message` en la cola del usuario (si es que existiera alguno) y elimina el mensaje de la misma.

En el siguiente ejemplo, la vista guarda un mensaje para el usuario después de crear una lista de reproducción:

```
def create_playlist(request, songs):  
    # Create the playlist with the given songs  
    request.user.message_set.create(  
        message="Your playlist was added successfully."  
    )  
    return render_to_response("playlists/create.html",  
        context_instance=RequestContext(request))
```

Al usar `RequestContext`, los mensajes del usuario actual, si los tuviera, están accesibles desde la variable de contexto usando el nombre `{{ messages }}`. El siguiente ejemplo representa un fragmento de código que muestra los mensajes:

```
{% if messages %}
<ul>
    {% for message in messages %}
    <li>{{ message }}</li>
    {% endfor %}
</ul>
{% endif %}
```